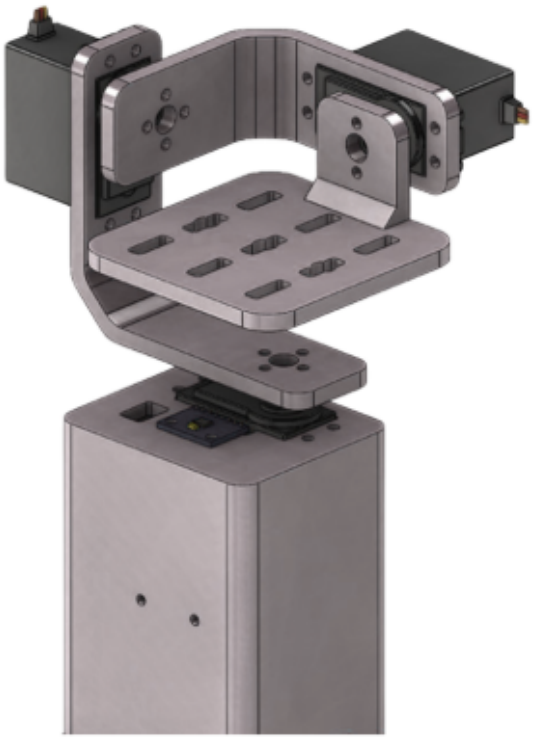
भारतीय प्रौद्योगिकी संस्थान कानपुर
Indian Institute of Technology Kanpur



High-Precision Self-Leveling Stabilizer

A System for Real-Time Actuator Stabilization

Comprehensive Technical Report

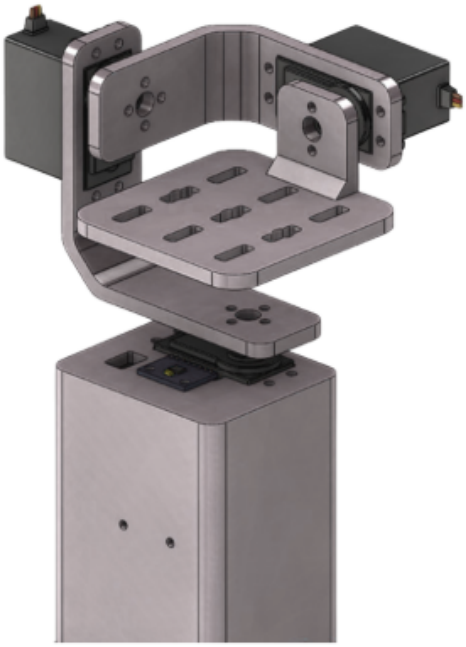
Project Members:
Akshat, Akshat Garg, Aryan Chaturvedi, Ayush Sharma,
Pathak Prince, Rahul Das

Department:
Materials Science and Engineering
Indian Institute of Technology Kanpur

April 15, 2026

Contents

1 Summary & Problem Statement



Maintaining stable orientation despite external vibrations and motion is a critical requirement in fields such as medical transport, drone navigation, and camera stabilization. This project develops a high-precision self-leveling stabilizer utilizing an ATmega328P-based Arduino Uno microcontroller and an MPU6050 Inertial Measurement Unit (IMU).

The system detects platform tilt by acquiring real-time 16-bit raw acceleration and angular velocity data. It then calculates the spatial orientation using a complementary filter and actively corrects deviations by actuating MG996R high-torque servo motors in the opposite direction of the detected tilt.

2 Mathematical Foundation & Sensor Fusion

The MPU6050 tracks 6-axis motion (3-axis accelerometer and 3-axis gyroscope). However, estimating pure orientation requires fusing these two distinct data streams to mitigate their individual mechanical and mathematical limitations.

2.1 Accelerometer Kinematics

The accelerometer estimates tilt by measuring the static force of gravity. The pitch or roll angle derived from accelerometer data, denoted as θ_{acc}, is calculated using the inverse tangent of the z-axis and y-axis acceleration vectors:

$$\theta_{acc} = \tan^{-1}\left(\frac{A_z}{A_y}\right) \tag{1}$$

While stable over long periods without accumulating drift, this method is highly susceptible to high-frequency noise and sudden mechanical vibrations.

2.2 Gyroscope Dynamics

The gyroscope measures the rate of angular velocity (ω). The current angle (θ_{gyro}) is determined by integrating the angular velocity over a time step (dt) and adding it to the previous angle (θ_{prev}):

$$\theta_{gyro} = \theta_{prev} + \omega \cdot dt \tag{2}$$

This provides a very smooth and immediate response to rapid motion but accumulates integration error (drift) over time, eventually skewing the baseline.

2.3 The Complementary Filter Algorithm

To achieve optimal accuracy, the system employs a Complementary Filter. This fusion algorithm applies a high-pass filter to the gyroscope data and a low-pass filter to the accelerometer data:

$$\theta_{fused} = \alpha \cdot (\theta_{prev} + \omega \cdot dt) + (1 - \alpha) \cdot \theta_{acc} \tag{3}$$

Where α (typically $0.96 \leq \alpha \leq 0.98$) dictates the trust weight between the short-term precision of the gyroscope and the long-term reliability of the accelerometer.

3 Core System Architecture

The stabilizer operates on a continuous, closed-loop feedback mechanism. The architecture is divided into sensing, processing, and actuation modules, as illustrated in Figure 1.

4 Hardware Specifications & Power Distribution

Stable power regulation is vital to prevent MCU brownouts caused by the high current draw of the three MG996R servo motors under load.

4.1 Bill of Materials (BOM) & Actual Billed Cost

The following table, shown in Table 4.1, reflects the actual components procured and their final billed costs (including applicable taxes, shipping, and pay-on-delivery fees), replacing the initial estimations. Items such as the Arduino microcontroller and 3D printing filament were sourced from existing laboratory inventory.

4.2 Circuit Pin Mapping Logic

To isolate control logic from mechanical power surges, power is distributed independently:

- **Power Plane:** The 7.4V battery pack connects to the Buck Converter. The converter's 5V out and Ground independently supply the Arduino, MPU6050, and all Servos.
- **Data Plane:** MPU-6050 utilizes I2C, connecting `SCL` to Arduino `A5`, `SDA` to `A4`, and `INT` to `D2`.

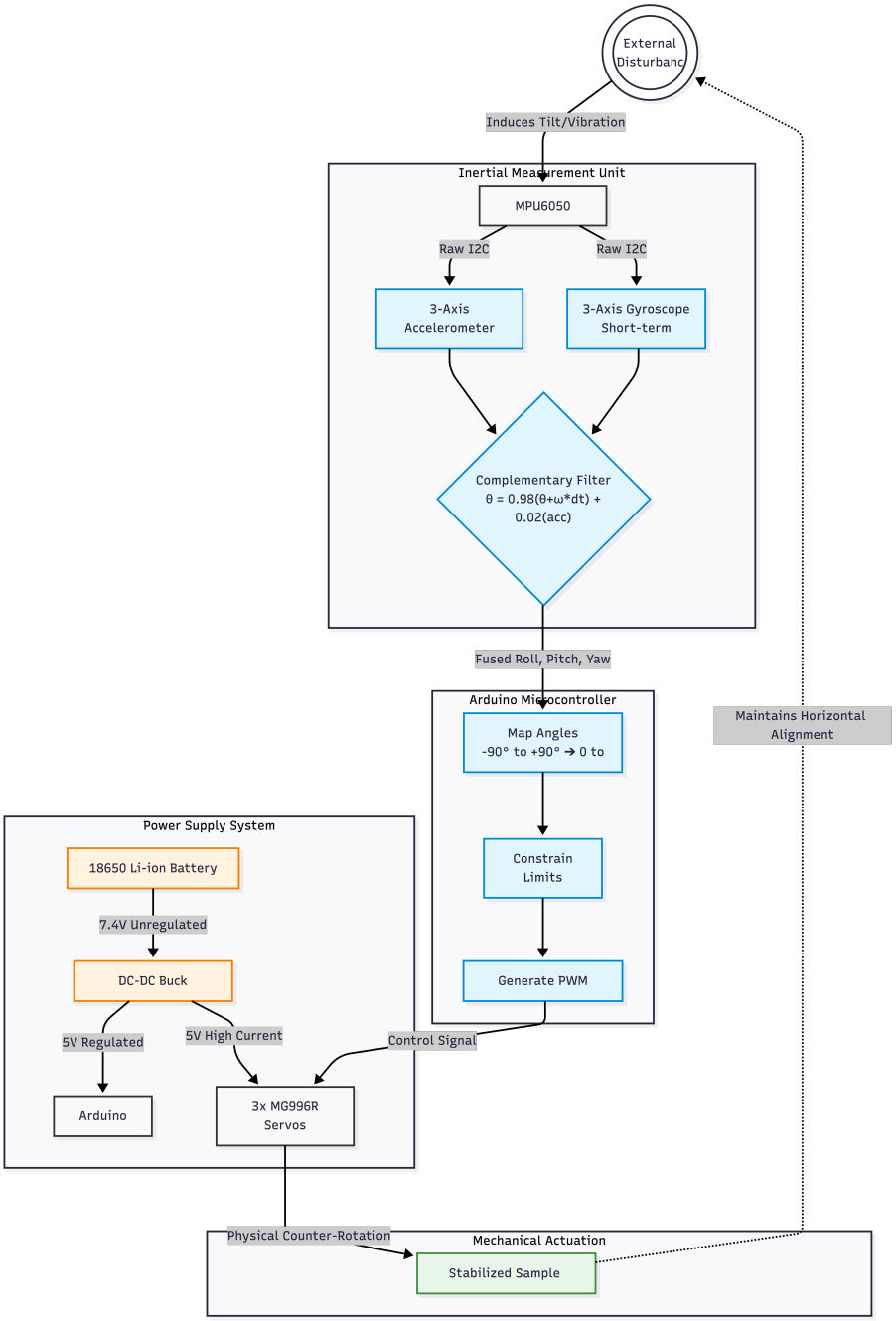


Figure 1: System Architecture

Component	Specification	Qty	Actual Cost (INR)
Servo Motors	TowerPro MG996R High Torque	3	1,048.80
Battery Pack	7.4V 2500mAH 18650 Li-ion	1	444.00
Fasteners	M3 Stainless Steel Nut/Bolt Set	1	281.30
MPU6050 IMU	3-Axis Gyro & Accelerometer	1	271.20
Buck Converter	LM2596 DC-to-DC Step-Down	1	165.70
Arduino Uno	ATmega328P MCU	1	*Lab Inventory*
Chassis Frame	PLA Filament (3D Printed)	1	*Lab Inventory*
		Total Procured Cost:	**2,211.00**

Table 1: Actual Hardware Components and Procured Cost (Invoices dated 18.02.2026)

- **Control Plane:** Arduino transmits PWM signals to the Roll Servo on `D9`, Pitch on `D8`, and Yaw on `D10`.

5 Software Implementation & Control Logic

5.1 Embedded Control Loop (Arduino)

The embedded control system operates in a continuous loop, where the MPU6050 IMU provides real-time orientation data. The system initializes communication over the I2C protocol, performs sensor calibration to eliminate bias, and continuously updates yaw, pitch, and roll angles. These angles are then linearly mapped to servo motor positions to achieve active stabilization.

Working Principle:

- The IMU provides orientation in terms of Yaw, Pitch, and Roll.
- These angles lie in the range $[-90°, 90°]$.
- They are mapped to servo limits $[0°, 180°]$.
- The servos actuate in real-time to counter platform tilt.

Embedded Control Logic using MPU6050 (C++)

```
#include "Wire.h"
#include <MPU6050_light.h>
#include <Servo.h>

// Create MPU object and Servo objects
MPU6050 mpu(Wire);
```

```
Servo myServo1; // Pitch
Servo myServo2; // Roll
Servo myServo3; // Yaw

void setup() {
  Serial.begin(9600);
  Wire.begin();

  // Initialize MPU6050 sensor
  byte status = mpu.begin();
  if (status != 0) {
    Serial.println("MPU6050 connection failed!");
    while (1) {} // Stop execution if sensor fails
  }

  // Attach servos to digital pins
  myServo1.attach(8);
  myServo2.attach(9);
  myServo3.attach(10);

  // Calibrate IMU (keep system stable during this)
  Serial.println("Calibrating...");
  delay(1000);
  mpu.calcOffsets(true, true);
  Serial.println("Calibration complete.");
  delay(1000);
}

void loop() {
  // Update sensor readings
  mpu.update();

  // Extract orientation angles
  float yaw   = -mpu.getAngleZ();
  float pitch = -mpu.getAngleY();
  float roll  =  mpu.getAngleX();

  // Map angles (-90 to 90) -> (0 to 180)
  int rollAngle  = map(roll,  -90, 90, 0, 180);
  int pitchAngle = map(pitch, -90, 90, 0, 180);
  int yawAngle   = map(yaw,   -90, 90, 0, 180);

  // Constrain to avoid mechanical limits
  rollAngle  = constrain(rollAngle, 0, 180);
  pitchAngle = constrain(pitchAngle, 0, 180);
  yawAngle   = constrain(yawAngle, 0, 180);
```

```
  // Actuate servos
  myServo2.write(rollAngle);
  myServo1.write(pitchAngle);
  myServo3.write(yawAngle);

  // Debug output (for visualization / serial monitor)
  Serial.print("R: "); Serial.print(roll);
  Serial.print(" P: "); Serial.print(pitch);
  Serial.print(" Y: "); Serial.println(yaw);
}
```

Key Observations:

- The use of `mpu.update()` ensures continuous real-time orientation tracking.
- Calibration (`calcOffsets`) is critical to eliminate sensor bias and drift.
- Linear mapping enables direct conversion from physical tilt to actuator response.
- Constraining angles prevents servo damage due to over-rotation.

5.2 Real-Time Visualization (Processing IDE)

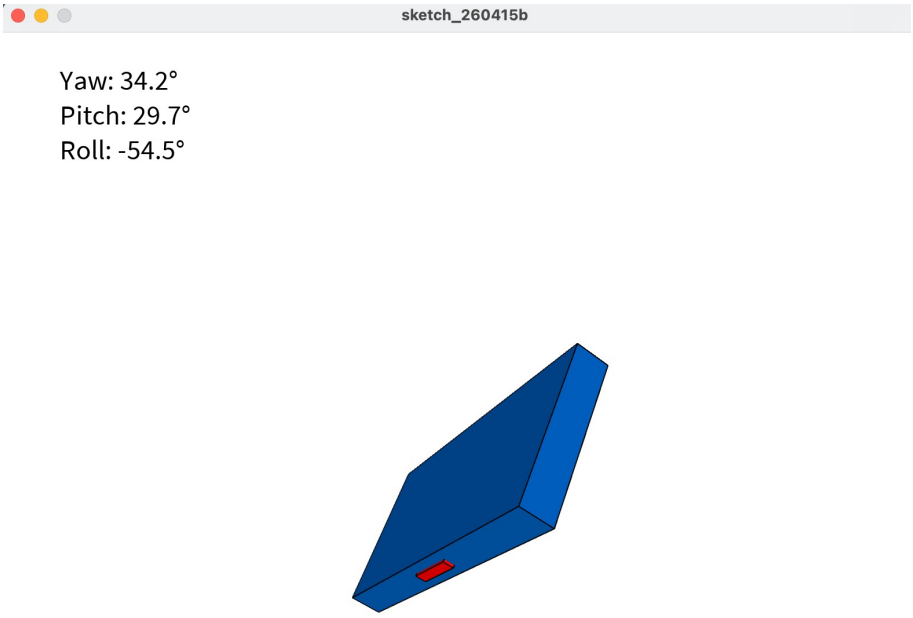


Figure 2: Real-Time 3D Visualization of Stabilizer Orientation

To enable real-time monitoring, the system transmits orientation data (yaw, pitch, roll) over a serial interface to a Processing-based graphical interface. The Processing environment utilizes the `P3D` renderer to generate a 3D visualization that acts as a digital twin of the physical stabilizer.

The received data is parsed and mapped to rotation transformations using functions such as `rotateX()`, `rotateY()`, and `rotateZ()`, allowing the virtual model to mimic the exact orientation of the hardware system.

Processing Visualization Code:

Processing IDE Code for 3D Visualization

```
import processing.serial.*;

Serial myPort;
float yaw, pitch, roll;

void setup() {
  size(800, 800, P3D); // Enable 3D rendering

  // Display available ports (for debugging)
  printArray(Serial.list());

  // Select the correct serial port (adjust index if needed)
  String portName = Serial.list()[0];
  myPort = new Serial(this, portName, 115200);
  myPort.bufferUntil('\n');
}

void draw() {
  background(30);
  lights();

  pushMatrix();
    translate(width/2, height/2, 0);

    // Apply rotations based on IMU data
    rotateX(radians(-pitch));
    rotateZ(radians(roll));
    rotateY(radians(-yaw));

    // Draw main body (platform)
    stroke(255);
    fill(0, 100, 200);
    box(300, 30, 150);

    // Front direction indicator
    fill(255, 0, 0);
    translate(0, 0, 75);
    box(50, 10, 5);
  popMatrix();

  // Display numerical angle values
```

```
  fill(255);
  textSize(24);
  text("Yaw: " + nfc(yaw, 1) + "°", 50, 50);
  text("Pitch: " + nfc(pitch, 1) + "°", 50, 80);
  text("Roll: " + nfc(roll, 1) + "°", 50, 110);
}

// Read serial data from Arduino
void serialEvent(Serial myPort) {
  String input = myPort.readStringUntil('\n');
  if (input != null) {
    input = trim(input);
    String[] values = split(input, '/');

    if (values.length == 3) {
      yaw = float(values[0]);
      pitch = float(values[1]);
      roll = float(values[2]);
    }
  }
}
```

Key Observations:

- The visualization provides an intuitive understanding of system orientation in real time.
- Serial communication ensures continuous data streaming from the microcontroller.
- The 3D model accurately reflects physical motion, validating sensor fusion and control logic.

6 Mechanical Design (CAD)

The structure was modeled in CAD and fabricated using PLA-based FDM 3D printing. The modular components include:

- **Bottom Cover:** Houses the core electronics, battery pack, and buck converter.
- **Yaw Servo Mount:** Provides the base for horizontal panning rotation.
- **Roll & Pitch:** Orthogonally mounted servos to maintain strict horizontal leveling.
- **Top Platform:** The final stabilized mounting plate to secure payloads.

Figure 3: Enter Caption

7 Conclusions & Future Scope

7.1 Results

The system successfully validates the concept of active tilt compensation using a low-cost, off-the-shelf component stack. The complementary filter dramatically reduced accelerometer noise while preventing gyroscope drift, enabling smooth and highly precise horizontal alignment.

7.2 Future System Enhancements

To further improve system performance, scalability, and robustness, several enhancements can be incorporated:

1. **PID-Based Closed-Loop Control:** Implementing a Proportional-Integral-Derivative (PID) controller will enable smoother stabilization with reduced steady-state error and minimal overshoot, improving dynamic response.

2. **Cartesian Coordinate-Based Control:** Instead of directly controlling servo angles in Euler space (Yaw-Pitch-Roll), the system can be extended to operate in a Cartesian coordinate framework. By defining the platform orientation in 3D space, inverse kinematics can be used to compute precise actuator commands. This approach enables smoother and more intuitive control, especially for complex stabilization tasks.

3. **Base-Mounted Feedback Sensor:** Introducing an additional sensor mounted on the stabilized platform (payload side) can significantly improve sensitivity and

accuracy. This creates a dual-feedback system where:

- The primary IMU measures input disturbances.
- The secondary sensor measures actual platform response.

This enables more accurate error correction and improved disturbance rejection.

4. **Brushless Balancing System:** Replacing servo motors with Brushless DC (BLDC) motors can significantly reduce latency, improve precision, and provide smoother actuation.
5. **Wireless Telemetry (IoT Integration):** Integration of an ESP32 module will allow real-time wireless monitoring, remote tuning, and cloud-based data logging.
6. **Sensor Upgrade (9-DOF IMU):** Upgrading to a 9-DOF IMU (e.g., MPU9250) with magnetometer support will improve yaw accuracy and reduce drift.
7. **Kalman Filter Implementation:** Replacing the complementary filter with a Kalman filter can provide optimal state estimation under noisy conditions.
8. **Auto-Tuning Algorithms:** Implementing adaptive control or auto-tuning PID gains will enhance system robustness under varying loads.